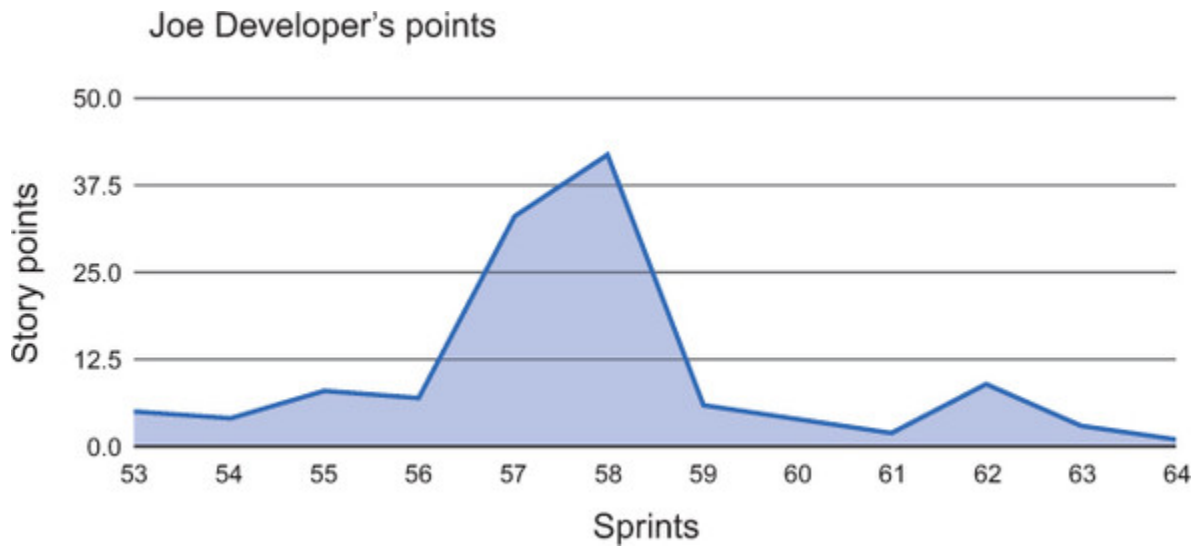
decides to pull data from all the developers across all projects to see if there's someone in particular who needs help with their work. Lo and behold, they find that Joe Developer, a member of Team 1, is off the charts during some of the dips, as shown in figure 1.15. After digging a bit deeper, it is determined that Joe Developer has specific knowledge of the product a different team was working on and was actually contributing to multiple teams during that time. He was overworked, yet it looked as if the team were underperforming.

Figure 1.15. Joe Developer's contribution is off the charts, but his team is not performing well.



In this case it's pretty easy to show that the commitment to multiple projects hurts the individual teams, and apparently Joe Developer is way more productive when working on the other team's product. People are usually eager to show trends like this, especially when they indicate that individual performance is great but for some reason forces outside the team are hurting overall productivity. If you're being measured, then you might as well ensure that the entire story is being told. In this case the data ended up pointing to a problem that the team could take action on: fully dedicate Joe Developer to one team to help avoid missing goals on another team and to not burn your engineers as you're delivering product.

Collecting data is so easy that you shouldn't need buy-in from product sponsors to get it started. Remember that collecting metrics needs to be cheap, so cheap that you shouldn't have to ask for permission or resources